

Exercicio 1: Construción da imaxe de produción para Laravel

Explicación Agora que temos xa o arquetipo de Laravel, vamos coller xa un código implantado para ver como podemos realizar a posta en produción dunha aplicación realizada con este *framework*. Deberemos tamén lanzar unha migración de base de datos, debido a que a aplicación necesita un esquema concreto para o seu funcionamento.

1. Crea un `fork` do proxecto `Arquetipo de Laravel`

e clónao no teu equipo de nome `T05.02 Despegue Laravel`

.

Descarga o seguinte [código](#) e inclúeo no directorio `src` (copia ben tódolos ficheiros ocultos, senón podes obter problemas).

Comproba que podes levantar o proxecto sen problemas con `make up`

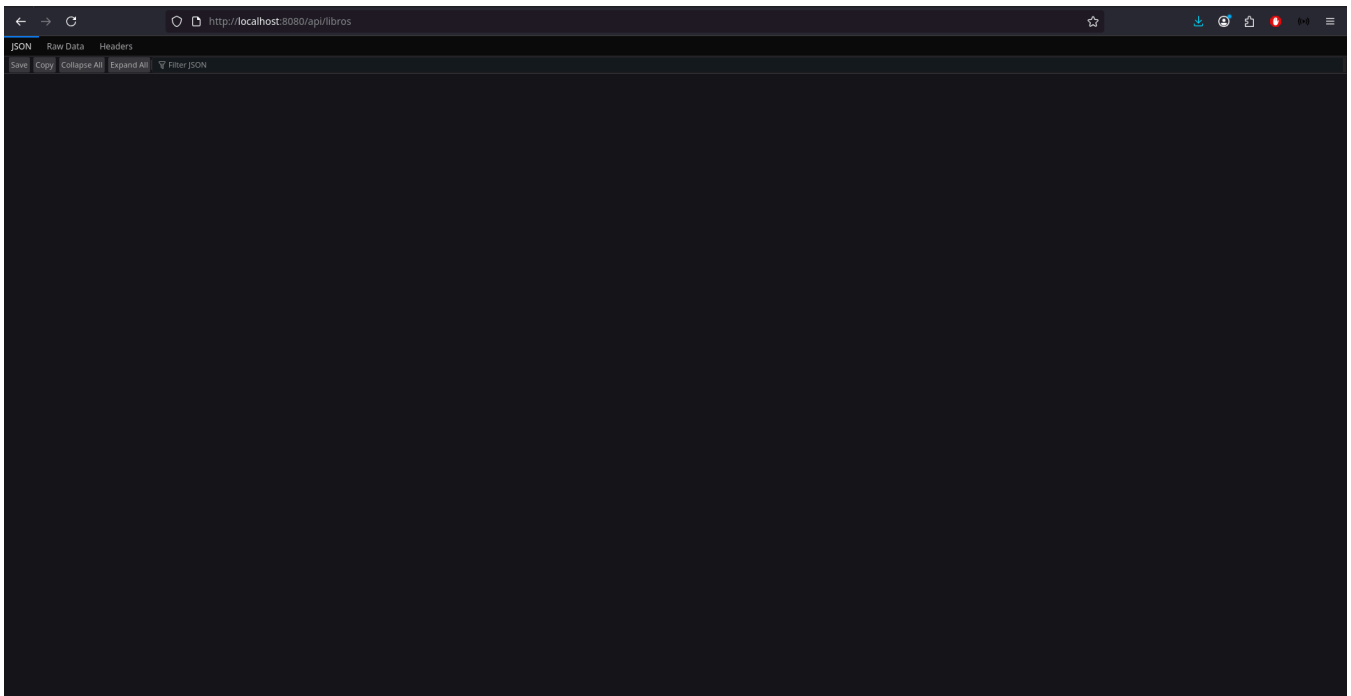
visitando `localhost:8080`

.

Realiza a migración. Despois visita a URL para comprobar que non hai erros

`http://localhost:8080/api/libros`

e que a aplicación funciona correctamente. **Realiza capturas** do funcionamento da aplicación.



Elimina o directorio `vendor`

1. do directorio `src`.

Explicación Agora procederemos a poñer desenvolver o procedemento de posta en produción da aplicación. Esta posta en produción realizarémola en Docker.

1. Crea o `Dockerfile`

de produción de nome `Dockerfile.prod`

. Recorda instalar as librerías necesarias para conectar con PostresSQL.

Xera unha `APP_KEY`

para o proxecto (esta inclúe a parte de `base64`

1.) utilizando o terminal:

```
APP_KEY="base64:${head -c 32 /dev/urandom | base64}"  
echo $APP_KEY
```

1. Crea o o ficheiro `docker-compose.prod.yaml`

e `entrypoint.prod.sh`

1. para produción. Modifica tan só as variables de conexión a base de datos polas que tiñamos en Aiven.
Mapea o servizo da aplicación para que escoite polo **porto 9001**.

Explicación Podemos crear novos comandos `make` para poder probar a posta en produción da nosa aplicación. Neste caso as probas podémolas facer no propio equipo e comprobar que todo funciona correctamente.

1. Crea un novo comando `deploy`

para despregar en produción e outro `deploy-stop`

1. para parar a produción:

```
# E interesante o --build para que se reconstrúa a imaxe sempre  
deploy:  
    docker compose -f docker-compose.prod.yaml up -d --build  
  
deploy-stop:  
    docker compose -f docker-compose.prod.yaml down
```

1. Pon en marcha a aplicación en produción e comproba que funciona a URL: <http://localhost:9001/api/libros>.
Entrega capturas dos ficheiros `Dockerfile.prod`

```

1 DockerFile.prod
2 ##### Etapa 1: Builder #####
3 FROM composer:2.6 AS builder
4 WORKDIR /app
5
6 # Copiamos ficheros de dependencias e instalamos
7 COPY src/ ./
8 RUN composer install --no-dev --optimize-autoloader --no-scripts
9
10
11 ##### Etapa 2: Imaxe final con Apache #####
12 FROM php:8.2-apache
13
14 # Instalar extensións necesarias para Laravel
15 RUN apt-get update && apt-get install -y \
16     libzip-dev \
17     zlib1g-dev \
18     libpq-dev \
19     && docker-php-ext-install zip pdo_pgsql pgsql \
20     && rm -rf /var/lib/apt/lists/*
21
22 # Directorio de traballo
23 WORKDIR /var/www/html
24
25 # Copiar aplicación desde o builder
26 COPY --from=builder /app /var/www/html
27
28 # Cambiar permisos para usuario non root
29 RUN chown -R www-data:www-data /var/www/html
30
31 # Copiamos a definición do virtualhost
32 COPY laravel.conf /etc/apache2/sites-available/000-default.conf
33
34 # Habilitar mod_rewrite de Apache
35 RUN a2enmod rewrite
36
37 # Copiamos o entrypoint
38 COPY entrypoint.prod.sh /usr/local/bin/entrypoint.sh
39 RUN chmod +x /usr/local/bin/entrypoint.sh
40 ENTRYPOINT ["usr/local/bin/entrypoint.sh"]
41
42 RUN echo "ServerName localhost" >> /etc/apache2/apache2.conf
43
44 # Cambiamos ao usuario de Apache
45 USER www-data
46
47 # Comando por defecto: Apache en primeiro plano
48 CMD ["apache2-foreground"]

```

, docker-compose.prod.yaml,

```

1 docker-compose.prod.yaml
2 services:
3   app:
4     build:
5       context: .
6       dockerfile: DockerFile.prod
7     environment:
8       APP_ENV: production
9       APP_DEBUG: false
10      APP_KEY: kgVDA0ptBa76DCftNh4vyRQ+517kiwQpxQEmg6BpsE=
11      SESSION_DRIVER: file
12      DB_CONNECTION: pgsql
13      DB_HOST: pg-36a04b08-iessanclemente-864d.j.aivencloud.com
14      DB_PORT: 10378
15      DB_DATABASE: defaultdb
16      DB_USERNAME: avnadmin
17      DB_PASSWORD: AVNS_RdVZ0aSiUWzeIuEGGjv
18    restart: always
19    ports:
20      - "9001:80"
21    networks:
22      - laravel_prod
23
24 networks:
25   laravel_prod:

```

entrypoint.prod.yaml

```
$ entrypoint.prod.sh
1  #!/bin/bash
2  set -e
3
4  APP_DIR="/var/www/html" # directorio raíz de Apache no contenedor
5  cd "$APP_DIR"
6
7  # Copiar .env.example a .env se non existe
8  if [ ! -f ".env" ]; then
9      cp .env.example .env
10 fi
11
12 # Sobrescribir tódalas variables de contorno relevantes no .env
13 # Filtra só variables típicas de Laravel
14 env | while IFS='=' read -r key val; do
15     case "$key" in
16         APP_*|DB_*|SESSION_*|CACHE_*|QUEUE_*|MAIL_*|LOG_*)
17             if grep -q "^$key=" .env; then
18                 sed -i "s/^$key=.*/$key=$val/" .env
19             else
20                 echo "$key=$val" >> .env
21             fi
22         ;;
23     esac
24 done
25
26 # Creamos directorios que Laravel necesita e non sempre crea
27 mkdir -p storage/framework/cache \
28         storage/framework/sessions \
29         storage/framework/views \
30         bootstrap/cache
31
32 # Limpar cache de configuración para asegurar que Laravel use as variables actualizadas
33 php artisan config:clear
34 php artisan cache:clear
35 php artisan view:clear
36
37 # Creamos as novas caches de configuración
38 php artisan config:cache
39 php artisan route:cache
40 php artisan view:cache
41
42 # Finalmente, deixar que Apache arranque
43 exec "$@"
```

e Makefile

```

M Makefile
1  # Variables. Define o nome do contedor da aplicación. Por iso é importante o valor container_n
2  APP = laravel_app
3
4  ##### Tarefas
5
6  # Levantar contedores. Executa "docker compose up" en modo segundo plano (`-d`). Levanta tódol
7  up:
8      docker compose up -d
9
10 # Parar contedores. Para e elimina os contedores levantados.
11 down:
12     docker compose down
13
14 # Entra no contedor da aplicación. Abre unha sesión interactiva ("bash") dentro do contedor `l
15 bash:
16     docker exec -it $(APP) bash
17
18
19 # Executar migracións. Executa as migracións de Laravel dentro do contedor. Mantén a base de d
20 migrate:
21     docker exec -it $(APP) php artisan migrate
22
23
24 # Instalar unha librería específica. para executar: "make install-lib LIB_NAME=monolog/monolog
25 install-lib:
26     docker exec -it $(APP) composer require $(LIB_NAME)
27
28 # E interesante o --build para que se reconstrúa a imaxe sempre
29 deploy:
30     docker compose -f docker-compose.prod.yaml up -d --build
31
32 deploy-stop:
33     docker compose -f docker-compose.prod.yaml down

```

1..

2. Para o despregue, e agora sube ao repositorio tódolos cambios.

Explicación Podemos engadir os ficheiros xerados para a posta en produción ao noso arquetipo de Laravel. Deste xeito, cando creemos unha nova aplicación con este *framework* tamén teremos realizado o procedemento de posta en produción.

1. Agora no repositorio `Arquetipo de Laravel`

podes engadir o novo `Makefile` e os ficheiros de `Dockerfile.prod`, `docker-compose.prod.yaml` e `entrypoint.prod.yaml`

.

No repositorio `Arquetipo de Laravel`

crea un ficheiro `Readme.md` explicando en que consiste o arquetipo na súa totalidade: que contén, como funciona, para que serve, etc. **Entrega capturas** da totalidade do ficheiro `Readme.md`

```

1  README.md > ## # Arquetipo de Laravel > ## Para que sirva
2  # Arquetipo de Laravel
3
4  Este repositorio contém un **arquetipo de aplicación Laravel**.
5
6  ## Contido
7
8  O arquetipo inclúe:
9
10 - Estrutura base dun proxecto Laravel.
11 - Configuración para conexión a PostgreSQL.
12 - Ficheiros para poata en produción (Todos os .prod).
13 - Makefile con comandos útiles.
14
15 ## Como funciona
16
17 1. **Desenvolvemento**
18   - Levanta os contedores en modo desenvolvemento.
19   - Podes acceder á aplicación en 'http://localhost:8080'.
20
21 2. **Producción**
22   - Modifica 'docker-compose.prod.yaml' coas credenciais de base de datos reais.
23   - Levanta a aplicación de produción.
24   - Podes acceder á aplicación en 'http://localhost:9001'.
25
26 ## Para que sirva
27
28 Pódese empregar como base para outros proxectos de laravel, so tendo que amiar a información dentro dos docker compose sobre a base de datos empregada.

```

1..

Exercicio 2: Despregue da aplicación de Laravel en produción

Explicación Vamos preparar a nosa máquina de produción (`dapw_iniciais_server_01`) para poder utilizar os ficheiros `Makefile`

. Tamén instalaremos Docker, debido a que agora nesta máquina a produción realizarase mediante esta tecnoloxía.

1. Na máquina `dapw_iniciais_server_01`

recupera a instantánea `admin_remota_configurada`

.

Instala o paquete `build-essential`

para poder utilizar os ficheiros `Makefile`

.

Instala Docker: <https://docs.docker.com/engine/install/debian/>. E realiza a configuración post instalación (<https://docs.docker.com/engine/install/linux-postinstall>). E dicir, engade o usuario `dadmin`

ao grupo `docker`. **Entrega captura** do `Hello world` de Docker. (Non é necesario `Docker rootless`

)

```

dadmin@iniciaisserver01:~$ docker run hello-world
Unable to find image 'hello-world:latest' locally
latest: Pulling from library/hello-world
17eec7bbc9d7: Pull complete
ea52d2000f90: Download complete
Digest: sha256:05813aedc15fb7b4d732e1be879d3252c1c9c25d885824f6295cab4538cb85cd
Status: Downloaded newer image for hello-world:latest

Hello from Docker!
This message shows that your installation appears to be working correctly.

To generate this message, Docker took the following steps:
1. The Docker client contacted the Docker daemon.
2. The Docker daemon pulled the "hello-world" image from the Docker Hub.
   (amd64)
3. The Docker daemon created a new container from that image which runs the
   executable that produces the output you are currently reading.
4. The Docker daemon streamed that output to the Docker client, which sent it
   to your terminal.

To try something more ambitious, you can run an Ubuntu container with:
$ docker run -it ubuntu bash

Share images, automate workflows, and more with a free Docker ID:
https://hub.docker.com/

For more examples and ideas, visit:
https://docs.docker.com/get-started/

```

Apaga a máquina e crea unha nova instantánea `docker_instalado`

. Crea ademais unha `OVA` de nome `dapw-docker.ova`

1. É importante esta última porque necesitarémola máis adiante.

Explicación A partir das seguintes tarefas utilizaremos varias máquina e vai ser importante ter claro que IP ten cada unha delas. Polo tanto é conveniente non utilizar o servidor DHCP para isto, e utilizar IPs fixas nos servidores. Senón dependeríamos ata da secuencia de inicio das máquinas para que se repetisen as IPs.

Esta máquina a partir deste momento converterase no **servidor de produción con Docker**.

1. Deshabilita o servidor DHCP da rede `Anfitrión sólo anfitrión`

Inicia a máquina `dapw_iniciais_server_01`

e modifica o ficheiro `/etc/network/interfaces` para asignar unha IP estática a máquina na rede de `Anfitrión sólo anfitrión`

- 1.:

```

auto enp0s8
iface enp0s8 inet static
address 192.168.56.101
netmask 255.255.255.0

```

1. Executa o comando `sudo systemctl restart networking.service`

2. e comproba con `ip a` que obtiveches a IP desexada.

Explicación Para finalizar a tarefa, tan só necesitamos o noso repositorio para despregar en produción a nosa aplicación. Docker permítenos reproducir de xeito exacto a posta en produción. Polo tanto se funcionou no noso equipo, vai funcionar en calquera servidor.

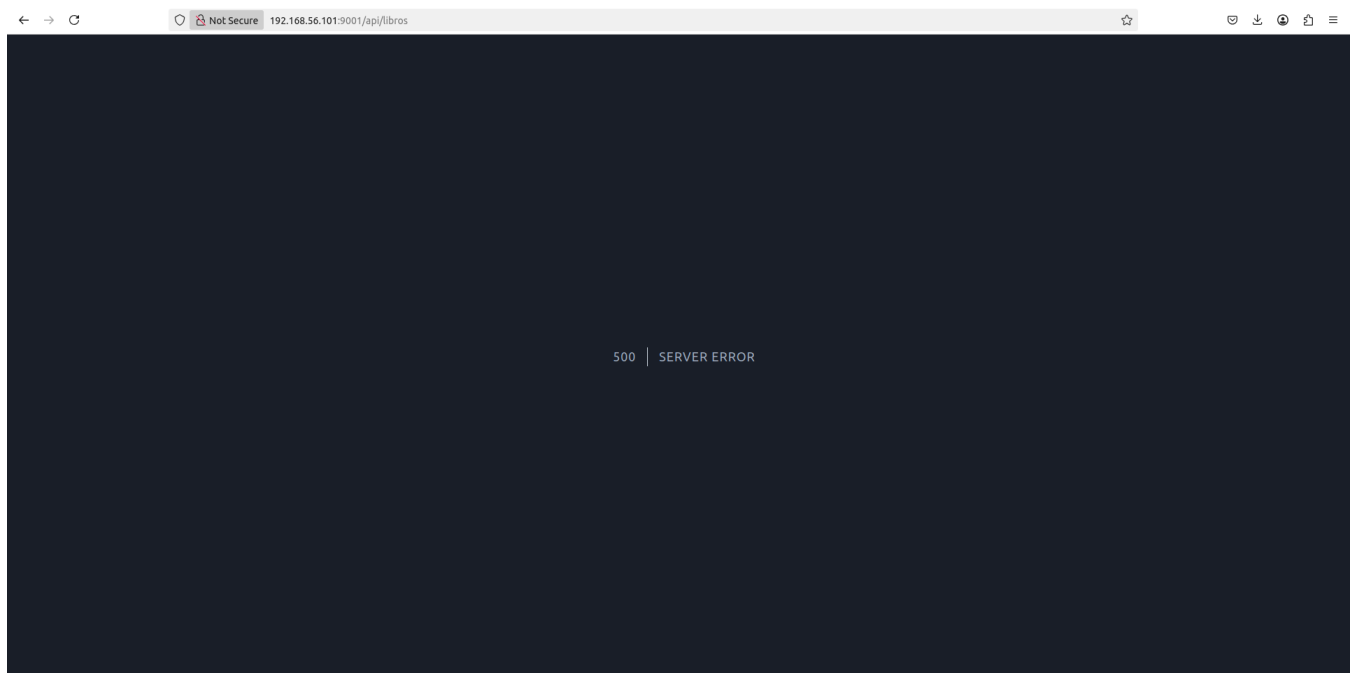
1. Inicia a máquina `dapw_iniciais_server_01`

. Clona o repositorio `T05.02 Despegue Laravel`

e executa o despegue.

Proba que todo funciona utilizando a seguinte URL: `http://192.168.56.101:9001/api/libros`

. **Entrega captura** onde se vexa a aplicación funcionando.



Exercicio 3: CI/CD para a construción da imaxe de produción

Explicación En GitLab.com podemos engadir os nosos propios `runners`. Isto é moi interesante porque podemos facer que os `runners`

teñan acceso a nosa propia rede e polo tanto acceder aos servizos e recursos desta. Para iso primeiro crearemos unha máquina virtual a partir das que xa temos.

1. Importa a OVA que creamos no paso anterior. Cando a importes preme en `Configuración`

e en `Política de dirección MAC` selecciona `Generar una nueva dirección MAC para todos los adaptadores de red` para que se xeren novas MACs. Chámalle a máquina `dapw_iniciais_server_02`

.

Inicia a máquina e internamente modifícalle o nome por `iniciaiserver02`

. Posteriormente reinicia.

Modifica o ficheiro `/etc/network/interfaces`

para asignar unha IP estática a máquina na rede de `Anfitrión sólo anfitrión`

1.:

```
auto enp0s8
iface enp0s8 inet static
address 192.168.56.102
netmask 255.255.255.0
```

1. Executa o comando `sudo systemctl restart networking.service`

1. e comproba con `ip a` que obtiveches a IP desexada.

2. Conéctate mediante SSH dende VSC.

Explicación A continuación instalaremos a aplicación que permite crear `runners`

de GitLab. E importante que o servizo que se instala poida utilizar `DOcker`. Polo tanto debemos engadir o usuario do servizo ao grupo de `Docker`

.

1. Vamos a instalar o paquete que permite crear `runners`

de GitLab (<https://docs.gitlab.com/runner/install/linux-repository/>).

Mete ao usuario `gitlab-runner`

no grupo de `Docker`

1. .

Explicación Vamos crear o noso primeiro `runner`

. Hai varios tipos de `runner`. Neste caso crearemos un `runner` de `shell`. Utilizaremos `tags` para despois poder seleccionar este `runner`

.

Un `runner de shell`

é un programa que executa os `jobs` dun `pipeline` **no propio servidor ou máquina onde está instalado**, usando a **consola do sistema** (`bash` no noso caso) para executar os comandos definidos no `.gitlab-ci.yml`

.

Os `tags nos runners`

serven para **controlar qué `jobs` poden executarse en cada `runner`**. Son etiquetas que permiten **asignar `jobs` específicos a determinados `runners`**

.

Ao rexistrar un `runner`

, podes poñerlle `tags`. Despois no `.gitlab-ci.yml` podes indicar que `tags` necesita un `job`. No seguinte exemplo vemos un `job` que se vai executar nun `runner` que conteña a etiqueta `console`

.

1. Vaite ao repositorio `T05.02 Despegue Laravel`

na interface web. Acude a `Ajustes > CI/CD` e desprega a lapela `Runners`

.

Preme en `Crear ejecutor de proyecto`

e ponlle como etiqueta `shell-docker` e preme en `Crear un runner del proyecto`

.

Executa na máquina `iniciaiserver02`

o código que che da para crear o `runner`

1. no paso 1. Selecciona as seguintes opcións:

- `URL` : deixa a que ven por defecto, é dicir non poñas nada.
- `Name` : `shell-runner-1`

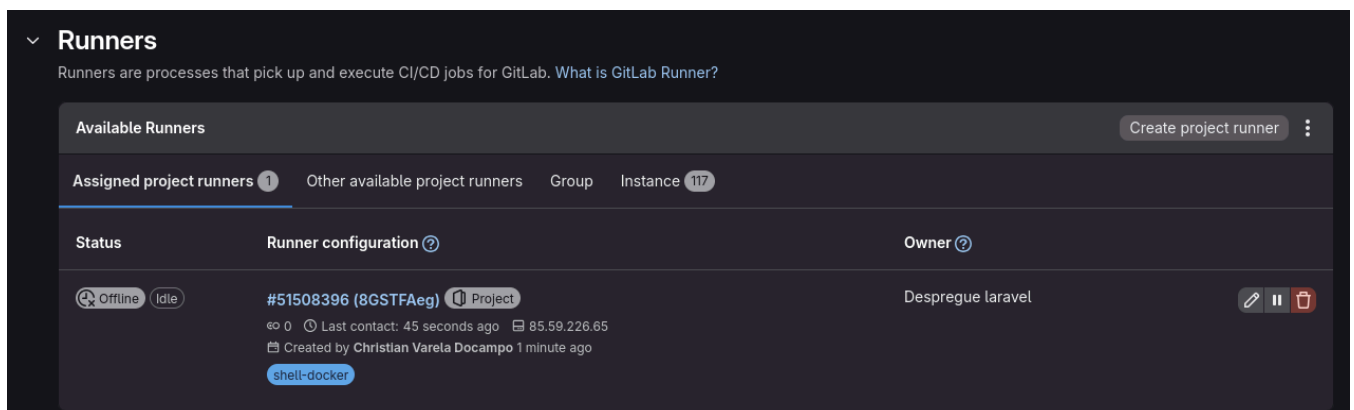
.

Executor

- `: shell`.

1. Acude de novo `Ajustes > CI/CD`

e desprega a lapela `Runners` . Verás agora que tes o `runner` xa configurado. **Entrega capturas** do `runner`



1. .

Explicación Agora crearemos un segundo `runner`

de tipo `Docker` . Este tipo de `runners`

foron os que estivemos realizando en tarefas anteriores.

Un `runner de executor Docker`

é un `runner` que executa os `jobs` dun `pipeline`

dentro de contedores Docker en lugar de usar o `shell` do sistema. Por iso era necesario que estes puideran utilizar Docker.

Cada `job` corre nun **contedor limpo** que se crea especificamente para el. Permite **illar o ambiente** de compilación e dependencias. Facilita usar diferentes imaxes Docker por `job`, garantindo reproducibilidade.

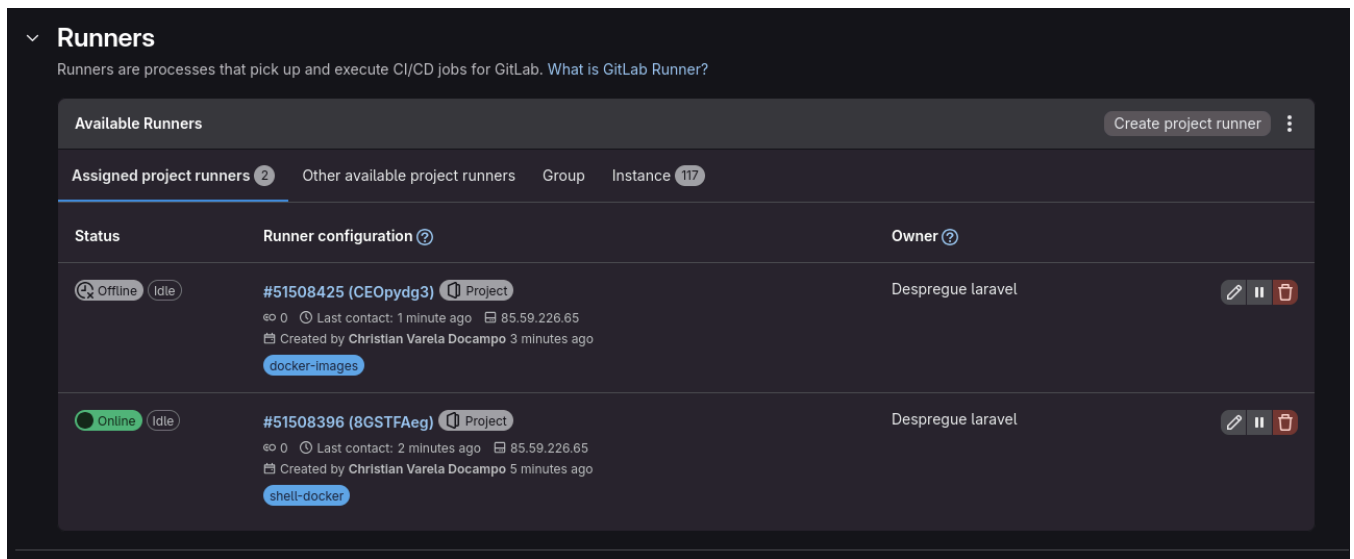
1. Agora crea outro `runner`

de `Executor` Docker, de nome `decker-runner-1` e coa etiqueta `docker-images`. Como imaxe por defecto introduce `debian`

.

Acude de novo `Ajustes > CI/CD`

e desprega a lapela `Runners`. Verás agora que tes o novo `runner` xa configurado. **Entrega capturas** do `runner`



.

Executa `gitlab-runner run`

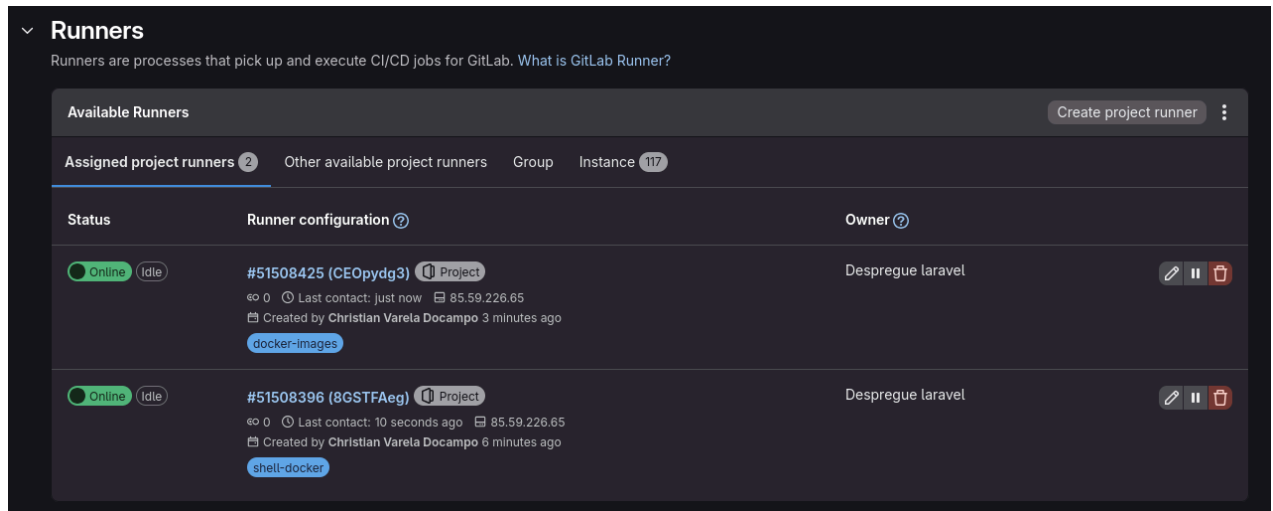
para que se executen os `runners`

.

Acude de novo `Ajustes > CI/CD`

e desprega a lapela `Runners`. Verás agora que tes os dous `runners` en funcionamento. **Entrega capturas** do `runner`

1..



Senón tes verificada a conta en GitLab.com, para que funcione debes desactivar os runners do propio Gitlab. Acude a lapela Instance e deshabilita Activar ejecutores de instancia en este proyecto

Explicación Poderemos utilizar os runners

creados para poder facer un build das nosas imaxes de Docker e subilas a un rexistro de imaxes como DockerHub

Un rexistro de imaxes Docker é un **repositorio onde se almacenan e distribúen imaxes Docker**. Serve para:

- Gardar imaxes construídas (con aplicacións, dependencias, configuración).
- Descargar imaxes para executar contedores en calquera máquina.
- Facilitar o **compartir e reutilizar contedores** entre equipos ou proxectos.

Docker Hub

é o **rexistro público oficial de Docker**. Permite **subir, baixar e compartir imaxes Docker**. Contén **imaxes oficiais** (como nginx, node, ubuntu) e imaxes de terceiros.

1. Acude a DockerHub

e crea unha conta co correo do IES San Clemente.

Acude en DockerHub

no teu perfil a Account Settings. Despois a Personal access token e preme en Generate new token. Dalle os permisos de Read, Write, Delete

1. . Garda este token que che xera porque non poderás volver a velo.

2. Crea un ficheiro de definición de CI/CD no teu repositorio que (faino no equipo anfitrión):

- Teña dúas etapas build e deploy

Na etapa de `build` crea un `job` que:

- Selecciona mediante a directiva `tag` o os `runners`

coa etiqueta `shell-docker`

- .

`tags:`

```
- shell-docker
```

- Utiliza o comando `docker-build`

para construír unha imaxe. Utiliza como etiqueta `usuarioDockerHub/laravel:latest`

Inicia sesión en `DockerHub`

- :

```
- echo "o_te_uotoken" | docker login -u usuarioDockerHub --password-stdin
```

- Sube a imaxe a `DockerHub`

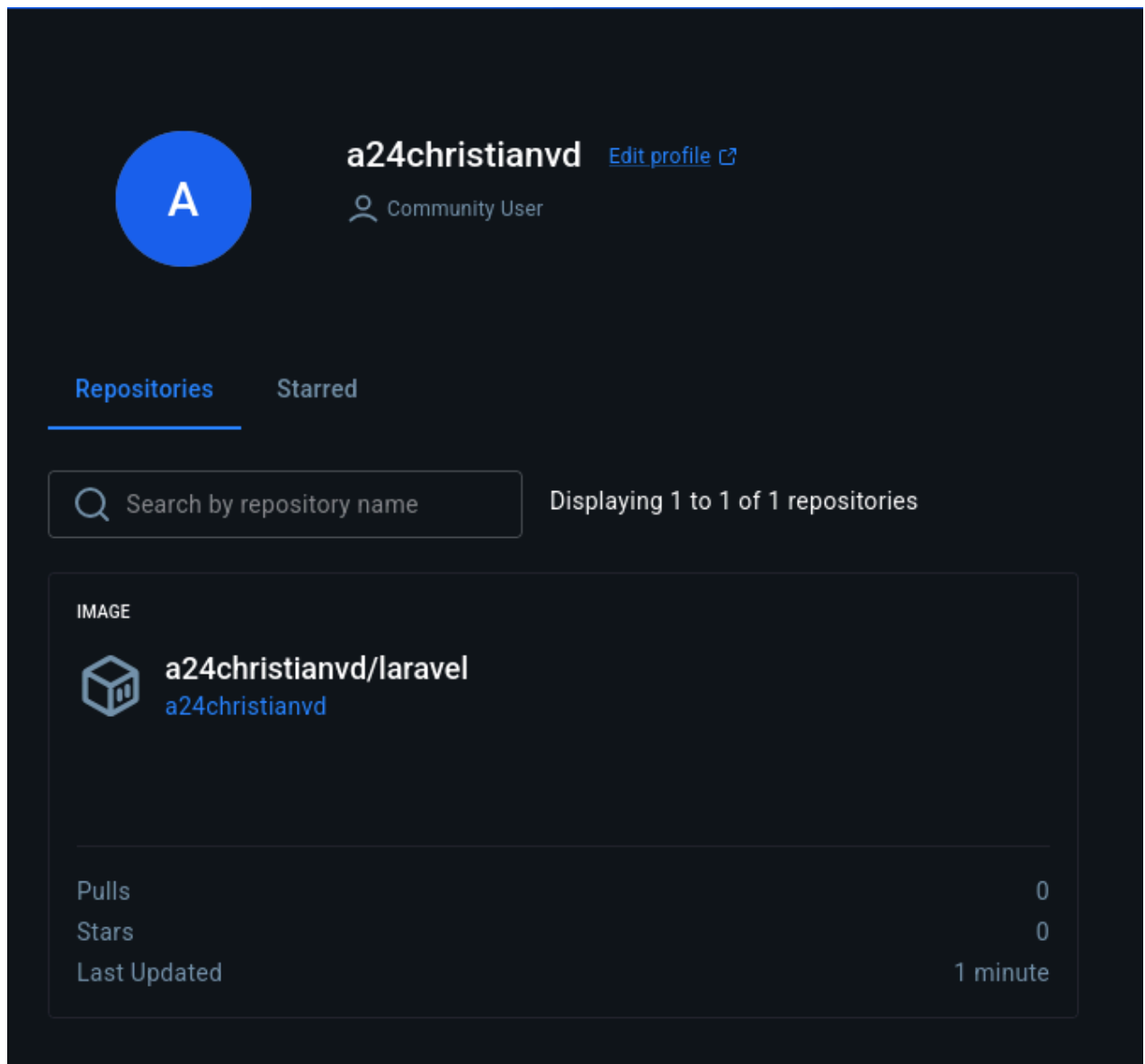
- o .

```
- docker push usuarioDockerHub/laravel:latest
```

- o Só se executará na rama `main`.

1. Realiza un `commit`

1. e `push` e a túa imaxe debería subirse a conta de DockerHub. **Entrega capturas** do ficheiro de definición de CI/CD e das túas imaxes en DockerHub.



Explicación Unha vez que temos a nosa imaxe nun rexistro de imaxes Docker, podemos modificar o noso `docker-compose.yaml`

de produción para que en lugar de facer o *build* da imaxe a descargue do rexistro.

En contornos de produción debemos utilizar esta opción e non facer o *build* no propio servidor.

Ademais podemos modificar o noso comando `deploy`

para que cando o executemos baixe se hai imaxes novas das nosas aplicacións e se hai algunha, pois rexenere os contedores con estas novas imaxes. Isto permítenos realizar dun xeito sinxelo actualizacións da aplicación, con tan só executar este comando.

1. Agora modifica o `docker-compose.prod.yaml`

do proxecto para que utilice a imaxe que creaches e subiches a DockerHub en lugar de reconstruír a imaxe. Engade que utilice sempre a última versión.

Modifica tamén o comando `deploy`

do `Makefile`

1. . Executa estes dous comandos no seu lugar:

```
docker-compose -f docker-compose.prod.yml pull
docker-compose -f docker-compose.prod.yml up -d
```

Explicación Vamos agora deixar preparado no noso servidor de produción ca última versión do proxecto, onde utilizamos as imaxes descargadas do rexistro.

1. En `iniciaisserver01`

para o despregue da aplicación se aínda o tes funcionando. Actualiza o repositorio e volve a realizar o `deploy`

Proba que todo funciona utilizando a seguinte URL: `http://192.168.56.101:9001/api/libros`

1..

Exercicio 4: Automatización de despregue da nova versión dunha aplicación

Explicación Instalaremos `webhook`

no servidor de produción de Docker que temos. Utilizando peticións HTTP poderemos indicarlle o servidor que execute diversas operacións.

No noso caso crearemos unha acción que permita mediante unha determinada petición HTTP actualizar o despregue da aplicación realizada en Laravel.

1. En `iniciaisserver01`

instalar `webhook`

En `/home/dadmin`

crea o ficheiro `laravel.sh`

1.. Neste *script*:

- Entra no directorio co contido do repositorio.
- Realiza un `pull` de `git`, para recuperar a versión máis recente de `git`.
- Executa o comando `deploy`
- de `make`.

1. Dalle permisos de execución ao *script* `laravel.sh`

1. para que poida ser executado.

Explicación No *script* anterior o primeiro que realizamos e actualizar o repositorio. Tal e como temos o repositorio, cada vez que queremos actualizalo teremos que meter as nosas credenciais.

Cando intentamos automatizar tarefas, como neste caso, o despregue de unha aplicación, necesitamos que os *scripts* non sexan interactivos. Polo tanto necesitamos que actualizar o repositorio de `git` non pida credenciais.

`Git` mediante **SSH con claves pública e privada** é un mecanismo de autenticación que permite:

- **Autenticación segura sen contrasinais.** O teu equipo ten unha **clave privada**, e o servidor `Git` coñece a túa **clave pública**. Cada vez que conectas, o servidor comproba que tes a clave correcta sen pedir contrasinal.
- **Integridade da conexión** A comunicación entre o teu cliente `Git` e o servidor está cifrada. Evita que alguén poida interceptar ou modificar os datos do repositorio mentres se transmiten.
- **Control de acceso.** Só os usuarios cuxas claves públicas están no servidor poden clonar, empurrar ou puxar cambios. Permite definir permisos finos para diferentes usuarios sen compartir contrasinais.

1. Crea un par de claves públicas e privadas SSH sen `passphrase`

1.:

```
$ ssh-keygen -t rsa -b 2048 -C "<comentario>"
```

1. Executa este comando `cat ~/.ssh/id_rsa.pub`

para obter a clave pública. Copia dita clave.

Vaite a `GitLab.com`. E vai a editar o teu perfil. Preme en `Claves SSH`

. E preme en `Add new key`

1. . Pega a clave pública utilizada anteriormente.

Explicación O repositorio que temos clonado no servidor de produción utiliza o protocolo HTTPS, polo tanto seguiremos pedindo credenciais. Debemos entón volver clonar o repositorio utilizando o protocolo SSH. Deste xeito tamén comprobaremos que todo funciona correctamente en canto a conexión de `git` mediante SSH.

1. Para o despregue de Laravel se é que se está executando. Borra o repositorio, e volve clonalo pero agora utilizando SSH. **Entre capturas** da execución do comando e da súa saída.

```
dadmin@iniciaisserver01:~$ git clone git@gitlab.com:a24christianvd/despregue-laravel.git
Clonando en 'despregue-laravel'...
remote: Enumerating objects: 191, done.
remote: Counting objects: 100% (25/25), done.
remote: Compressing objects: 100% (25/25), done.
remote: Total 191 (delta 10), reused 0 (delta 0), pack-reused 166 (from 2)
Recibiendo objetos: 100% (191/191), 920.18 KiB | 2.51 MiB/s, listo.
Resolviendo deltas: 100% (46/46), listo.
dadmin@iniciaisserver01:~$
```

Explicación Agora debemos crear o ficheiro que define a acción e a URL do `webhook`

. Entón poderemos executar a petición HTTP correspondente e que se actualice o repositorio.

1. Crea o ficheiro `hooks.json`

en `/home/dadmin`

1. no que:

- Se execute o *script* `laravel.sh`
- para que se realice a actualización do despregue.
- Pon un *token* xerado por esta web para gañar seguridade: <https://it-tools.tech/token-generator>.

1. Pon en marcha a aplicación dos `webhooks`

Dende o equipo anfitrión executa a chamada HTTP, pódelo facer mediante un comando `curl`. **Entrega capturas** da execución do comando `curl` e da saída do mesmo.

```
infe@debian:~/Documents$ curl -X GET http://localhost:9000/hooks/deploy-laravel?token=yQLz80IT4gURPM7h3k05jQvqb9YBS6eEa5yQCQ5R5AcjfaxiEf28A4y9S03QmGCD
Deploy triggeredinfe@debian:~/Documents$
```

Ademais na consola onde se está executando a aplicación `weebhook`

1. podes ver a saída producida pola execución da acción. **Entrega captura** disto.

```
dadmin@iniciasserver01:~$ webhook -hooks hooks.json -port 9000 -verbose
[webhook] 2026/01/28 23:32:53 version 2.8.0 starting
[webhook] 2026/01/28 23:32:53 setting up os signal watcher
[webhook] 2026/01/28 23:32:53 attempting to load hooks from hooks.json
[webhook] 2026/01/28 23:32:53 found 1 hook(s) in file
[webhook] 2026/01/28 23:32:53 loaded: deploy-laravel
[webhook] 2026/01/28 23:32:53 serving hooks on http://0.0.0.0:9000/hooks/{id}
[webhook] 2026/01/28 23:32:53 os signal watcher ready
[webhook] 2026/01/28 23:32:56 [2ac08f] incoming HTTP GET request from [::1]:36450
[webhook] 2026/01/28 23:32:56 [2ac08f] deploy-laravel got matched
[webhook] 2026/01/28 23:32:56 [2ac08f] error parsing body payload due to unsupported content type header:
[webhook] 2026/01/28 23:32:56 [2ac08f] deploy-laravel hook triggered successfully
[webhook] 2026/01/28 23:32:56 [2ac08f] 200 | 16 B | 74.501µs | localhost:9000 | GET /hooks/deploy-laravel?token=yQLz80IT4gURPM7h3k05jQvqb9YBS6eEa5yQCQ5R5AcjfaxiEf28A4y9S03QmGCD
[webhook] 2026/01/28 23:32:56 [2ac08f] executing laravel.sh (/home/dadmin/laravel.sh) with arguments ["laravel.sh"] and environment [] using /home/dadmin/ as cwd
```

Explicación Para finalizar, podemos facer que cando se cree unha nova imaxe para produción se actualice automaticamente a imaxe que está despregada, permitindo realizar unha automatización total.

Por iso necesitamos o `runner`

de `shell` executándose nunha máquina da nosa rede. Esta poderá realizar un comando `curl` que active o `webhook`

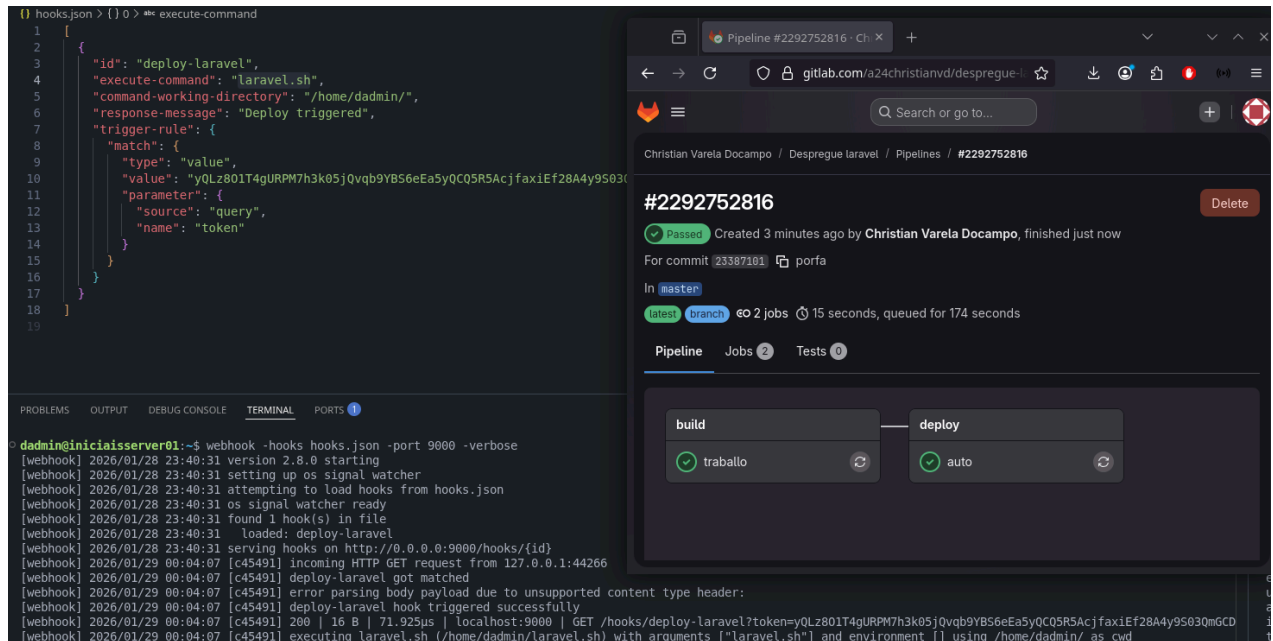
1. No equipo anfitrión, engade un `job` no repositorio para a etapa `deploy`

. Utiliza o `runner`

de `shell`. Este realizará unha chamada mediante `curl` para actualizar o despregue.

Comproba que se executou correctamente o *script* vendo a saída do comando `webhook`

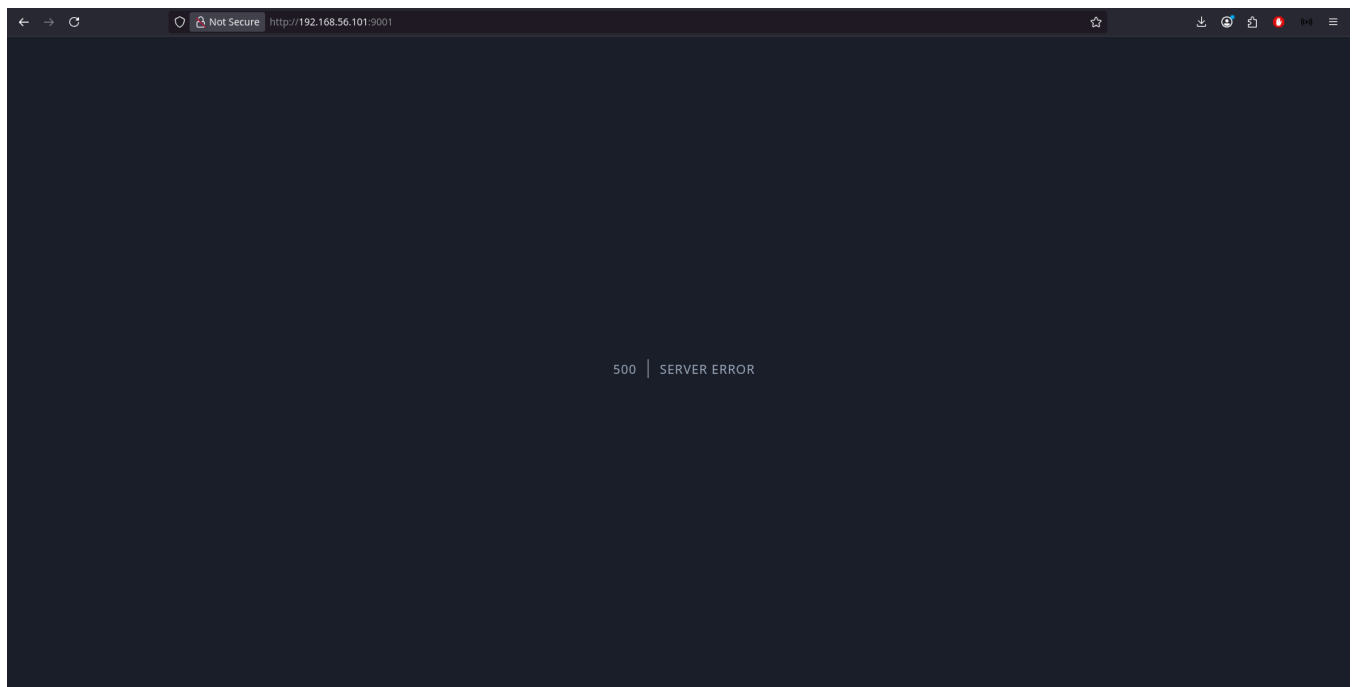
1. **Entrega capturas** desta comprobación.



Explicación Como exemplo final, vamos facer unha modificación na aplicación. Veremos como con só facer un **push** ao repositorio **git** se despregará a nova versión da aplicación de xeito automatico.

1. Abre a URL **http://192.168.56.101:9001**

. Verás a pantalla de inicio de Laravel. **Entrega captura** desta páxina.



Abre o repositorio no equipo anfitrión. Modifica o ficheiro **src/resources/views/welcome.blade.php**

. Onde pon **Let's get started**

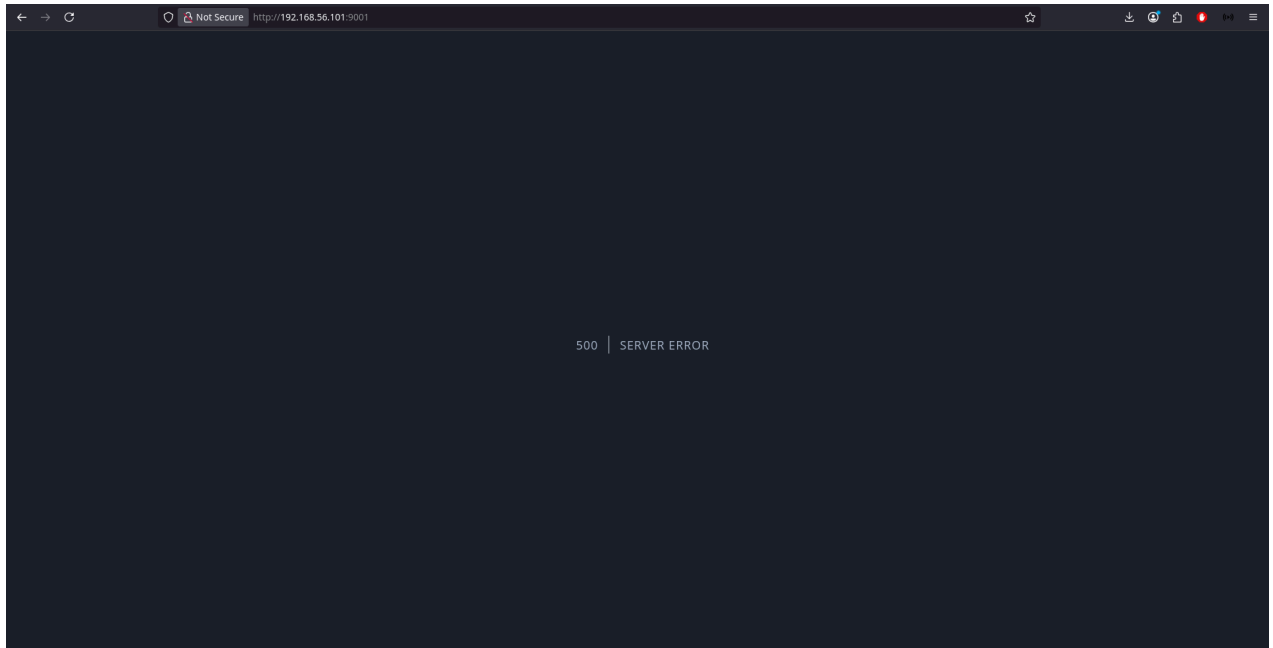
, cámbiao polo teu nome.

Fai un **commit**

e un **push** e espera a que despregue de novo a aplicación.

Abre a URL **http://192.168.56.101:9001**

1. . Verás a pantalla de inicio de Laravel. **Entrega captura** desta páxina.



Exercicio 5: Posta en produción dunha aplicación en Vue.js

Explicación A continuación vamos despregar a aplicación de Vue.js. Aquí tes o [código](#).

IMPORTANTE: Volvemos necesitar un `entrypoint.prod.sh`

porque Vue.js en produción non colle as variables de contorno, o que si fai no contorno de desenvolvemento. Polo tanto no noso `entrypoint.prod.js` imos crear un ficheiro `env.js` no que se almacenen ditas variables, e ademais engadir no `index.html` un enlace a ese ficheiro. Deste xeito o navegador descargará o ficheiro `env.js`

Sigue os seguintes pasos para realizar dita tarefa.

- **Paso 1:** Proba en local do funcionamento do despregue en produción.
 - Parte do arquetipo de Vue.js creado no anterior exercicio realizando un `fork`.
 - Crea un `Dockerfile.prod`

:

- Debe ser multi-etapa, onde na primeira traspiles o código e na segunda tan só o despregues con Apache.
- En Apache, se utilizas a imaxe oficial `httpd` cambian varios dos directorios e usuarios propietarios. Para facilitar podes utilizar unha imaxe de Debian e instalar Apache. Podes utilizar isto:

```
FROM debian:bullseye-slim

# Instalamos Apache e utilidades
RUN apt-get update && apt-get install -y \
    apache2 \
    libapache2-mod-rewrite \
```

```
&& apt-get clean \  
&& rm -rf /var/lib/apt/lists/*
```

Resto de código

Para a execución

```
CMD ["apache2ctl", "-D", "FOREGROUND"]
```

Crea o seguinte `entrypont.prod.sh`

- :

```
#!/bin/sh  
set -e  
  
# Rutas dos ficheiros  
HTML_PATH="/var/www/html/index.html"  
ENV_JS_PATH="/var/www/html/env.js"  
  
echo "Xerando env.js desde variables de contorno do docker-compose"  
  
echo "window.__ENV__ = {" > "$ENV_JS_PATH"  
  
FIRST=true  
for VAR in $(env | grep '^VITE_' | sort); do  
KEY=$(echo "$VAR" | cut -d= -f1)  
VALUE=$(echo "$VAR" | cut -d= -f2-)  
  
if [ "$FIRST" = true ]; then  
FIRST=false  
else  
echo ", " >> "$ENV_JS_PATH"  
fi  
  
# Escapar comiñas dobres  
ESCAPED_VALUE=$(printf '%s' "$VALUE" | sed 's/"/\\"/g')  
  
echo " $KEY: \"$ESCAPED_VALUE\" >> "$ENV_JS_PATH"  
done  
  
echo "" >> "$ENV_JS_PATH"  
echo "}" >> "$ENV_JS_PATH"  
  
echo "env.js xerado."  
  
echo "Inxectando <script src=\"/env.js\"></script> en index.html..."  
  
if ! grep -q 'src="/env.js"' "$HTML_PATH"; then  
# Inserta despois da liña <head> ou antes do primeiro <script type="module"  
sed -i '/<script type="module"/i \<script src="/env.js"></script>' "$HTML_PATH"  
echo "✅ Script env.js inxectado"  
else  
echo "Script env.js xa inxectado"
```

```
echo "❗ env.js xa estaba inxectado"
fi

exec "$@"
```

- No `docker-compose.prod.yaml`

:

- Fai un *build* de `Dockerfile.prod`

.

```
docker-compose.prod.yaml
```

tan só debes de incluír a URL da API (`http://IP_SERVER:9001`

) e as propias de Vue.js.

Mapea a aplicación ao porto 9002.

Copia o `entrypoint.prod.sh`

- e execútao do mesmo xeito que en Laravel.

Crea un comando no `Makefile`

para facer o `deploy`

.

Proba a despregar a aplicación con `deploy`

- e que todo funcione. Para ver os datos debes ter funcionando a aplicación de Laravel.
- Fai un *commit* e un *push* do repositorio.

Paso 2: Subida automática da imaxe de Docker a DockerHub.

- Define o ficheiro de CI/CD para que constrúa a imaxe e a suba a DockerHub.
- Modifica o `docker-compose.prod.yaml`
- para que agora en lugar de construír a imaxe utilice a que se subiu a DockerHub.
- Fai un *commit* e un *push* do repositorio.
- Comproba que podes despregar a aplicación en Vue.js no servidor de Docker.

Paso 3: Despregue no servidor de produción.

- Configura o `hooks.json`

para poder automatizar o despregue.

Engade no ficheiro de definición de CI/CD para que se realice o despregue automaticamente tras un `commit`

- - na rama `main`.
 - Comproba o correcto funcionamento.

Entrega capturas do ficheiro `Dockerfile.prod`

```
1 ##### ETAPA 1: BUILD DE VUE #####
2 FROM node:latest as node
3
4 # Directorio de traballo
5 WORKDIR /app
6
7 COPY package*.json ./
8
9 RUN npm install
10
11 COPY . .
12
13 RUN npm run build
14
15 ##### ETAPA 2: IMAXE FINAL CON APACHE #####
16 FROM debian:bullseye-slim
17
18 # Instalamos Apache e módulos necesarios
19 RUN apt-get update && apt-get install -y \
20 |     apache2 \
21 |     && apt-get clean \
22 |     && rm -rf /var/lib/apt/lists/*
23
24 # Directorio onde Apache serve os ficheiros
25 WORKDIR /var/www/html
26
27 # Copiamos o resultado do build de Vue
28 COPY --from=node /app/dist /var/www/html/public
29
30 RUN chown -R www-data:www-data /var/www/html
31
32 # Copiamos o entrypoint
33 COPY entrypoint.sh /usr/local/bin/entrypoint.sh
34 RUN chmod +x /usr/local/bin/entrypoint.sh
35 ENTRYPOINT ["/usr/local/bin/entrypoint.sh"]
36
37 # Habilitar mod_rewrite
38 RUN a2enmod rewrite
39
40 # Copiar o VirtualHost personalizado
41 COPY vue.conf /etc/apache2/sites-available/000-default.conf
42
43 RUN echo "ServerName localhost" >> /etc/apache2/apache2.conf
44
45 RUN mkdir -p /var/run/apache2 \
46 |     && chown -R www-data:www-data /var/run/apache2 \
47 |     && chmod 755 /var/run/apache2
48
49 # USER www-data
50
51 # Arranque do entrypoint e logo Apache en foreground
52 CMD ["apache2ctl", "-D", "FOREGROUND"]
53
```

, `docker-compose.prod.yaml`,

```

1  services:
2    vue-app:
3      image: a24christianvd/vue:latest
4      container_name: vue_app
5      environment:
6        VITE_API_URL: "http://192.168.56.101:9001"
7      ports:
8        - '9002:80'
9      restart: "always"
10     volumes:
11       - ./:/app
12
13

```

hooks.json,

```

{} hooks.json > {} 0 > {} trigger-rule > {} match > {} parameter
1  [
2    {
3      "id": "deploy-vue",
4      "execute-command": "vue.sh",
5      "command-working-directory": "/home/dadmin/",
6      "response-message": "Deploy triggered",
7      "trigger-rule": {
8        "match": {
9          "type": "value",
10         "value": "yQLz801T4gURPM7h3k05jQvqb9YBS6eEa5yQCQ5R5AcjfaxiEf28A4y9S03QmGCD",
11         "parameter": {
12           "source": "query",
13           "name": "token"
14         }
15       }
16     }
17   ]
18
19

```

.gitlab-ci.yml

```

vue > 🍌 .gitlab-ci.yml
1  stages:
2    - build
3    - deploy
4
5  trabajo:
6    stage: build
7    tags:
8    - shell-docker
9    script:
10   - docker buildx build -t a24christianvd/vue:latest .
11   - echo "dckr_pat_blu8xcK0eIIAqhIze6gQLIeEArk" | docker login -u a24christianvd --password-stdin
12   - docker push a24christianvd/vue:latest
13   only:
14     - master
15
16  auto:
17    stage: deploy
18    tags:
19    - shell-docker
20    script:
21     - echo "Disparando webhook para desplegar en producción..."
22     - curl -X GET http://localhost:9000/hooks/deploy-vue?token=yQLz801T4gURPM7h3k05jQvqb9YBS6eEa5yQCQ5R5AcjfaxiEf28A4y9S03QmGCD
23   only:
24     - master
25
26

```

e Makefile

```

vue > M makefile
1  APP = vue_app
2
3  up:
4    docker compose up --build
5
6  down:
7    docker compose down
8
9  bash:
10   docker exec -it $(APP) bash
11
12  install-lib:
13   docker exec -it $(APP) npm install $(LIB)
14
15  migrate:
16   docker exec -it $(APP) npm run build
17
18  deploy:
19   docker compose -f docker-compose.yml pull
20   docker compose -f docker-compose.yml up --build
21
22  deploy-stop:
23   docker compose -f docker-compose.yml down

```

Agora no repositorio Arquetipo de Vue.js

podes engadir o novo `Makefile`, e os ficheiros de `Dockerfile.prod`, `docker-compose.prod.yaml` e `entrypoint.prod.yaml`. Deste xeito para o próximo proxecto que desenvovas xa tes o proceso de produción realizado. No repositorio `Arquetipo de Laravel` crea un ficheiro `Readme.md` explicando en que consiste o arquetipo na súa totalidade.